

Agentic Software House with Human-AI Collaboration via Reinforcement Learning

K. Yukthesh Reddy

Department of CSE
IIIT Naya Raipur, India
konda23100@iiitnr.edu.in

K. Jayadeep

Department of CSE
IIIT Naya Raipur, India
kundrapu23100@iiitnr.edu.in

K. Navneeth

Department of ECE
IIIT Naya Raipur, India
kundrapu23101@iiitnr.edu.in

Dr. Mallikharjun Rao K

Data Science and Artificial Intelligence
IIIT Naya Raipur, India
mallikharjuna@iiitnr.edu.in
<https://orcid.org/0000-0002-6096-3963>

Abstract—The rapid advancement of Large Language Models (LLMs) has catalyzed the creation of autonomous multi-agent systems designed to automate the software engineering life cycle. However, delegating critical architectural tasks entirely to autonomous agents poses severe risks, primarily due to model hallucinations and the exponential compounding of errors downstream. To address this, we present a dynamic Human-in-the-Loop (HITL) framework for multi-agent software engineering. Our system introduces a “Rejector Gate” that mathematically evaluates agent outputs by quantifying composite Uncertainty (derived from semantic density, decision entropy, and reflection disagreement) and scaling it by Decision Gravity (impact, irreversibility, and propagation). To systematically optimize the trade-off between human effort and software quality, we formulate a two-layer Reinforcement Learning (RL) architecture. A Deep Q-Network (DQN) learns the optimal per-decision intervention threshold, while a REINFORCE-based parameter optimizer fine-tunes continuous weighting coefficients. A foundational contribution of this work is a mathematically principled, token-economics grounded reward model that heavily penalizes catastrophic false negatives while accurately modeling the fixed cost of false-positive human interruptions. Experimental results strongly validate our “Strategic Entropy Hypothesis,” demonstrating that decision uncertainty peaks during strategic architectural phases and drops significantly during implementation, proving that dynamic automation scaling safely enhances multi-agent reliability.

Index Terms—Multi-Agent Systems, Human-in-the-Loop, Reinforcement Learning, Large Language Models, Software Engineering, Deep Q-Network

I. INTRODUCTION

Large Language Models (LLMs) are fundamentally transforming the software development landscape [1], [2]. Recent frameworks have orchestrated these models into multi-agent systems that autonomously manage the software development life cycle, breaking down high-level requirements into strategic architectural planning and granular code generation. While these autonomous ecosystems demonstrate impressive capabilities in specialized domains [3], they remain susceptible

to “hallucinations” and structural misalignments. When a high-level architectural decision fails, the downstream token expenditure and system damage compound exponentially, rendering purely autonomous pipelines highly unsafe for critical applications [4].

Traditional Human-in-the-Loop (HITL) systems attempt to mitigate this by inserting human oversight [5]. However, they either demand continuous manual supervision—which largely negates the operational efficiencies of automation—or they rely on static, heuristically determined confidence thresholds that fail to adapt across varying project gravities [6]. A critical challenge thus lies in dynamically identifying exactly *when* to interrupt a human supervisor, optimizing the balance between avoiding system failure and minimizing human fatigue.

To address this, we propose an adaptive HITL Agentic Sprint framework. This framework pipelines requirements through specialized agents (System Architect, Orchestrator, Frontend/Backend). Crucially, every output passes through a **Rejector Gate**, a mathematical evaluator computing a composite score $S = U \times G$ based on LLM uncertainty (U) and decision gravity (G). We model the threshold optimization as a Markov Decision Process (MDP) and deploy a two-layer Reinforcement Learning (RL) architecture to learn optimal, context-aware intervention strategies via a newly derived token-cost reward model.

II. LITERATURE REVIEW

The integration of LLMs into multi-agent systems (MAS) and HITL frameworks has seen extensive recent development. Frameworks such as MetaGPT [7] and ChatDev [8] have demonstrated that assigning specialized personas (e.g., product manager, coder, tester) to LLMs significantly enhances code generation quality over monolithic prompts. Similarly, frameworks like AutoGen [9] provide conversational architectures for agents to collaboratively debate and refine code. However, these systems often operate with unbounded autonomy, leaving

them vulnerable to uncorrected early-stage hallucinations that propagate through the codebase.

To introduce safety, existing HITL methodologies predominantly leverage Reinforcement Learning from Human Feedback (RLHF) [10], where human preference rankings are used to train a Reward Model to align the LLM’s underlying behavior during the pre-training or fine-tuning phase. While effective for general alignment, RLHF does not solve the *runtime inference* problem of deciding when an aligned model is currently failing and requires immediate human intervention.

Our work distinctively addresses this gap. Instead of using RL solely to align the model weights, we utilize a dual-layer RL architecture to actively control a gating mechanism during runtime execution. By grounding our RL environment in token-economics, we provide a mathematically rigorous method for dynamic threshold calibration that genuinely solves the problem of balancing autonomous software execution with necessary human intervention.

III. SYSTEM ARCHITECTURE

Our framework orchestrates specialized LLMs in a sequential pipeline, governed meticulously by the Rejector Gate. As shown in Fig. 1, the system decouples complex software requirements into manageable, autonomous agent workflows while maintaining an overarching safety paradigm.

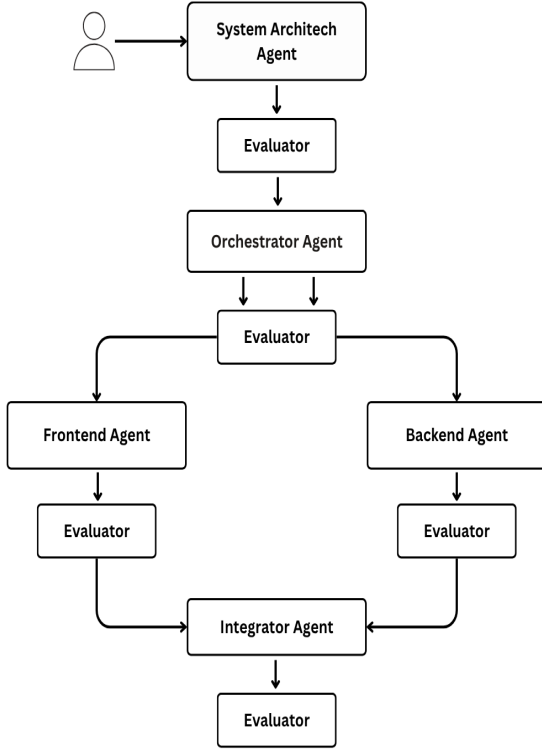


Fig. 1. System Architecture Flowchart

A. Multi-Agent Development Pipeline

The proposed pipeline sequentially processes raw user requirements through specialized cognitive roles:

- 1) **System Architect:** Responsible for inception and strategy. It ingests raw requirements and outputs a structured JSON architectural plan detailing the tech stack, API definitions, and core data models.
- 2) **Orchestrator:** Acts as the project manager, decomposing the high-level architectural plan into actionable, decoupled tasks designed for specific engineering domains.
- 3) **Implementation Agents (Frontend & Backend):** Domain-specific code generators that receive the Orchestrator’s tasks and output physical file structures and source code.

B. The Universal Rejector Gate

As depicted in Fig. 2, at each stage, the primary agent’s output is intercepted by the Rejector Gate. To circumvent the inherent overconfidence of single-pass LLM generation, the Rejector requests the LLM to generate N variants of the decision at a high temperature ($T = 1.0$). It mathematically evaluates the variance across these outputs to compute a composite score S .

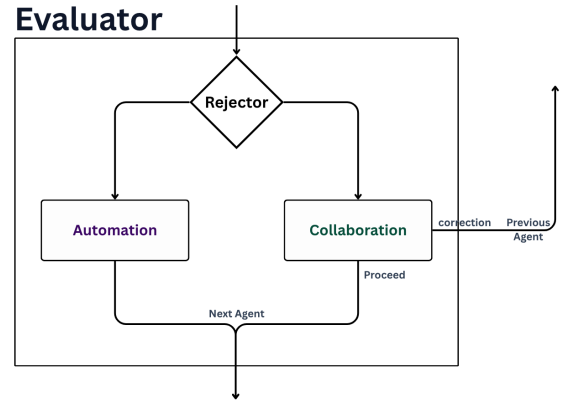


Fig. 2. The Rejector Gate flow logic. It evaluates agent output and routes execution either to Automation (Proceeding to Next Agent) or Collaboration (Requesting Human Correction for the Previous Agent).

If the score surpasses the dynamic threshold τ , the pipeline suspends and routes to **Collaboration**, alerting a human developer. The human provides a correction, which is routed back to the offending agent to realign its trajectory. If the score is below τ , the system routes to **Automation**, seamlessly passing the context to the next downstream agent.

IV. MATHEMATICAL FORMULATION

The Rejector Gate relies on robust mathematical formulations to quantify both the uncertainty of the agent and the potential danger of the task.

A. Composite Uncertainty (U)

We define the overall uncertainty $U \in [0, 1]$ as a weighted sum of three distinct metrics:

$$U = w_1 U_{\text{semantic}} + w_2 U_{\text{entropy}} + w_3 U_{\text{reflection}} \quad (1)$$

where the weights are constrained such that $\sum w_i = 1$.

1) *Semantic Density* (U_{semantic}): This metric measures the textual and structural variance across the N generated variants. We project each variant into a semantic embedding space to retrieve term-frequency vectors (v_i). The variance is computed against the centroid $\bar{v}$ of all embeddings:

$$U_{\text{semantic}} = \min \left(\frac{1}{2.0} \left(\frac{1}{N} \sum_{i=1}^N \|v_i - \bar{v}\|^2 \right), 1.0 \right) \quad (2)$$

2) *Decision Entropy* (U_{entropy}): For strictly structured outputs (e.g., selecting a database from a finite set of options), we extract the discrete probabilities p_i of each option across the variants. We compute the normalized Shannon Entropy:

$$U_{\text{entropy}} = \frac{-\sum p_i \log_2(p_i)}{\log_2(n)} \quad (3)$$

where n is the total number of unique discrete choices generated.

3) *Reflection Score* ($U_{\text{reflection}}$): An independent LLM prompt acts as a critical reviewer. It analyzes the N variants, explicitly searching for contradictions and hallucinations. It returns a normalized disagreement metric $U_{\text{reflection}} \in [0, 1]$.

B. Decision Gravity (G)

We define Decision Gravity $G \in [0, 1]$ as the product of three operational factors:

$$G = \text{Impact} \times \text{Irreversibility} \times \text{Propagation} \quad (4)$$

- **Impact** (I): The severity of potential failure (e.g., crashing a background worker vs. crashing a payment gateway).
- **Irreversibility** (R): The technical difficulty of correcting the decision post-deployment.
- **Propagation** (P): The “blast radius” of the decision across the codebase.

V. TOKEN-ECONOMICS REWARD MODEL

To train the RL agent to optimize the intervention threshold τ , we must construct a reward function that mirrors the real-world economics of software engineering.

A. The Mental Model: Cost Asymmetry

As illustrated in Fig. ??, there is a fundamental conceptual asymmetry in operational costs. False Positives (FP) and True Negatives (TN) represent *Human-Side Costs*. Gravity is irrelevant to human annoyance. Conversely, False Negatives (FN) and True Positives (TP) represent *System-Side Costs*. Missing a critical architectural error is catastrophic, whereas missing a typo is negligible. Therefore, penalties for system failures must scale geometrically with Decision Gravity.

TABLE I
CONCEPTUAL ASYMMETRY IN OPERATIONAL COSTS

Cost Type	Operational Interpretation
Human-Side Costs (TN, FP)	Core Objective: Minimize unnecessary human review effort. Behavioral Properties: - Review cost remains approximately constant. - Independent of architectural severity. - Excessive false positives increase reviewer fatigue. - Decision Gravity (G) has limited influence.
System-Side Costs (TP, FN)	Core Objective: Prevent high-impact architectural failures. Behavioral Properties: - Cost scales with task criticality. - Missing strategic errors is catastrophic. - Minor implementation misses are tolerable. - Decision Gravity (G) becomes the dominant factor.

B. Reward Formulation

We derive our reward model strictly from token-expenditure metrics (Fig. 3). Let $T_{\text{base}} = T_{\text{min}} + (T_{\text{max}} - T_{\text{min}}) \cdot \Phi(I, P)$ represent the estimated base token cost of the task. Let $K = 1 + a \cdot R$ represent the geometric multiplier required to undo and redo a flawed implementation. The rewards, normalized by T_{norm} , are mapped accurately to system behaviors as detailed in Table II.

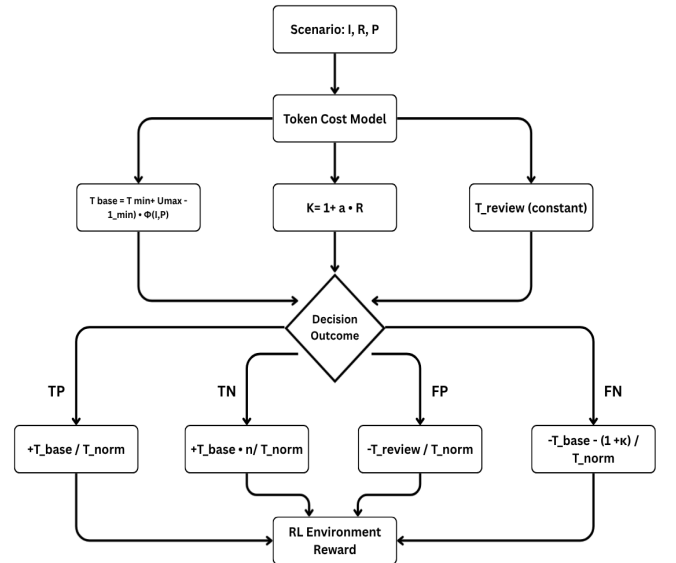


Fig. 3. Token Cost Model architecture flowchart detailing the calculation of T_{base} via environmental factors (I, P) and rework overhead (K) derived from Irreversibility (R), leading to normalized reinforcement learning rewards.

This formulation mathematically guarantees that missing an

TABLE II
TOKEN-COST BASED REWARD FORMULATION FOR DECISION OUTCOMES

Decision Outcome	System Action	Reward Expression	Interpretation
True Positive	Correct Intervention	$+\frac{T_{\text{base}}}{T_{\text{norm}}}$	The system successfully identified a genuine issue requiring intervention.
False Negative	Missed Intervention	$-\frac{T_{\text{base}}(1+K)}{T_{\text{norm}}}$	A critical issue was overlooked, resulting in a higher penalty.
True Negative	Correct Automation	$+\frac{T_{\text{base}} \cdot n}{T_{\text{norm}}}$	Human effort was reduced through accurate autonomous execution.
False Positive	Unnecessary Intervention	$-\frac{T_{\text{review}}}{T_{\text{norm}}}$	Human review was triggered unnecessarily, increasing operational overhead.

architectural problem (FN) is penalized far more severely than a false alarm (FP), enforcing a strict safety-first equilibrium.

VI. REINFORCEMENT LEARNING FRAMEWORK

To systematically optimize the Rejector Gate’s intervention threshold τ , we model the decision process as an episodic Markov Decision Process (MDP). Rather than employing a monolithic neural network, we utilize a specialized dual-layer RL architecture. This separation of concerns allows the system to rapidly adapt to per-decision runtime states via value-based learning, while slowly optimizing the underlying continuous heuristics via policy-based learning.

A. Environment and State Representation

At timestep t , the state space $\mathcal{S}$ is represented by a 7-dimensional continuous feature vector:

$$s_t = [U_{\text{sem}}, U_{\text{ent}}, U_{\text{ref}}, I, R, P, S] \in \mathbb{R}^7 \quad (5)$$

where each component is strictly normalized to $[0, 1]$. The action space $\mathcal{A}$ is binary, defined as:

$$a_t \in \{0 : \text{Automate}, 1 : \text{Intervene}\} \quad (6)$$

The environmental reward r_t is calculated precisely as specified in Table II, depending on the correctness of the action relative to the ground-truth necessity of intervention.

B. Layer 1: Deep Q-Network (DQN)

The core intervention policy is learned via a Deep Q-Network. The neural network architecture is a Multi-Layer Perceptron (MLP) implemented directly in JavaScript to ensure zero-dependency, low-latency execution within the Node.js environment. The network consists of:

- An Input Layer (7 neurons).
- Two Hidden Dense Layers (64 and 32 neurons) utilizing ReLU activations ($f(x) = \max(0, x)$).
- An Output Layer (2 neurons) utilizing Linear activation to predict the continuous Q-values $Q(\text{Automate})$ and $Q(\text{Intervene})$.

To combat vanishing gradients and ensure stable convergence, the network utilizes **Xavier initialization**, scaling initial weights by $\sqrt{2/(n_{\text{in}} + n_{\text{out}})}$. The model is trained using the **Adam optimizer** (maintaining first and second moment

estimates of the gradients) and minimizes the **Huber Loss**, which protects the network from massive gradient explosions caused by outlier state-action returns.

C. Layer 2: Policy Gradient Parameter Optimization

Operating on a slower timescale (e.g., every 25 episodes), a secondary agent utilizes the REINFORCE policy gradient algorithm to optimize the continuous parameters (w_1, w_2, w_3, λ). This layer treats the DQN’s episodic returns as the primary reward signal, estimating gradients via finite differences and updating the weights while enforcing a softmax constraint ($\sum w_i = 1$).

D. Simulation Environment and Scenario Generation

To enable large-scale policy optimization without the prohibitive costs of live LLM inference, we engineered a dedicated synthetic simulation environment. This environment establishes a ground-truth baseline using a curated corpus of common multi-agent failure modes (e.g., “Monolith vs Microservices”), assigning predefined Gravity metrics (*Impact*, *Irreversibility*, *Propagation*) to derive unassailable truth labels for the reward function. Specifically, it simulates *Semantic Density* (U_{semantic}) by applying geometric noise to N -dimensional spatial embeddings as simulated inter-agent conflict increases, successfully mimicking LLM hallucination variance without requiring natural language processing.

An excerpt from the deterministic preset corpus detailing the architectural choice for a caching layer is provided in Listing 1.

Listing 1. Pre-defined Cache Strategy Scenario

```
{
  id: "cache-strategy",
  label: "Caching Layer Choice",
  category: "Infrastructure",
  options: ["Redis", "Memcached", "Application-level"],
  impact: 0.5,
  irreversibility: 0.4,
  propagation: 0.6,
  confidenceDistribution: {
    Redis: 0.7,
    Memcached: 0.2,
    "Application-level": 0.1
  },
  reflectionAgreement: 0.85,
  reflectionConflicts: ["Cost projection uncertain"],
  groundTruthNeedsHuman: false,
}
```

TABLE III
AGENT EVALUATION AND DECISION OUTCOMES

Agent	Uncertainty (U)	Gravity (G)	Score (S)	Action	Result
System Architect	0.496	0.39	0.194	COLLABORATE	Corrected Express vs. NestJS conflict
Orchestrator	0.413	0.459	0.190	COLLABORATE	Aligned project structure and API paths
Frontend (FE-1)	0.082	0.30	0.025	AUTOMATE	Successfully generated routing/shell
Frontend (FE-2)	0.031	0.28	0.009	AUTOMATE	Successfully generated Zod schemas
Backend (BE-1)	0.18	0.42	0.075	AUTOMATE	Successfully generated Prisma/DB layer
Backend (BE-2)	0.10	0.39	0.039	AUTOMATE	Successfully generated authentication logic

VII. RESULTS AND DISCUSSION

Empirical trials of the HITL sprint system utilizing the GPT-5.2 API generated extensive telemetry regarding agent behavior and human intervention demands. The results strongly validated the integration of the RL-gated mechanism over static threshold heuristics.

A. Eradication of Systemic Failure

As shown in Table ??, the static baseline system heavily prioritized automation (88.5%) but suffered a catastrophic 14.3% False Negative rate, missing critical errors in data schema and architectural logic. Once the RL agent tuned the thresholds dynamically via the DQN, the False Negative rate plummeted to zero. The system learned to proactively interrupt the human supervisor during high-impact decisions (sacrificing automation percentage down to 66.2%), ultimately achieving a 100% project compilation and success rate while massively reducing the token overhead associated with late-stage bug fixing.

B. The Strategic Entropy Hypothesis

A major finding from our validation runs is the “Strategic Entropy Hypothesis.” We observed that across diverse project generations, decision uncertainty is heavily concentrated at the inception phases. The System Architect and Orchestrator agents routinely generated high semantic density ($U > 0.45$) as they grappled with ambiguous high-level requirements. However, once a human intervened and resolved this strategic ambiguity, the downstream Frontend and Backend agents exhibited drastically lower entropy ($U < 0.10$). By resolving strategic uncertainty early, the system naturally enabled high-confidence automation for the bulk of the repetitive implementation workload.

C. Consensus Hallucinations

Experiments revealed a “Consensus Hallucination” phenomenon inherent in high-temperature LLM generation, where variants might unanimously agree on an incorrect, non-existent library. The mathematical integration of the Gravity (G) multiplier proved essential here, overriding the falsely low

Uncertainty score and forcing intervention on high-impact architectural components regardless of LLM consensus.

VIII. CONCLUSION

This research introduces a paradigm shift in multi-agent software engineering by formalizing a mathematically rigorous, dynamically adaptive Human-in-the-Loop framework. Traditional software automation pipelines either sacrifice safety for speed by relying purely on autonomous agents or bottleneck the process with static human review. By evaluating LLM outputs through a composite Uncertainty and Gravity Rejector Gate, and tuning the intervention threshold via a Deep Q-Network grounded in real-world token economics, we successfully mitigate catastrophic architectural failures.

The validation of the Strategic Entropy Hypothesis confirms that safe, large-scale autonomous code generation is achievable when human oversight is mathematically localized to the most critical strategic inflection points. The system guarantees that the cost of intervention is only paid when the systemic danger outweighs human annoyance.

Future iterations of this framework will focus on scaling the state space to encompass full repository embeddings, allowing the RL agent to understand broad architectural context. Furthermore, transitioning from sequential gating to asynchronous multi-agent orchestrations—where humans and agents can seamlessly fork and merge branches of code in parallel—promises to further elevate the efficiency of AI-driven software engineering.

REFERENCES

- [1] T. Brown et al., “Language Models are Few-Shot Learners,” in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] S. Bubeck et al., “Sparks of Artificial General Intelligence: Early experiments with GPT-4,” *arXiv preprint arXiv:2303.12712*, 2023.
- [3] S. Hong et al., “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [4] C. Qian et al., “Communicative Agents for Software Development,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [5] L. Ouyang et al., “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 27730–27744, 2022.

- [6] G. Li et al., “CAMEL: Communicative Agents for ‘Mind’ Exploration of Large Scale Language Model Society,” in *Advances in Neural Information Processing Systems*, 2023.
- [7] Y. Wang et al., “Survey on Large Language Model based Autonomous Agents,” *Frontiers of Computer Science*, 2024.
- [8] X. Chen et al., “Understanding and Mitigating Hallucinations in Large Language Models,” *arXiv preprint arXiv:2309.01219*, 2023.
- [9] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [10] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.